



Full Stack Development Week 08

Topics covered:

- Full Stack Development using React, Node, Express and MySQL

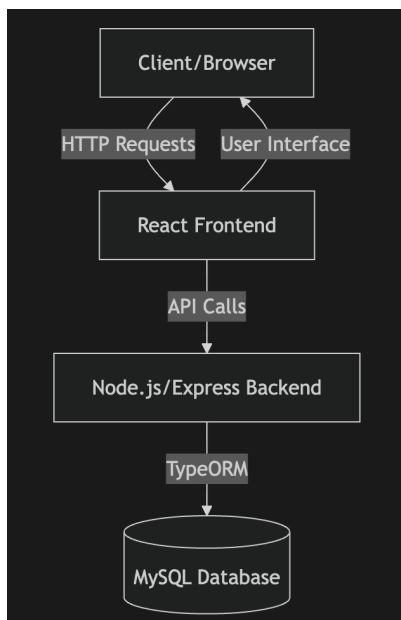
Why Full Stack Development

A full stack developer is someone who works across the entire web application stack - from frontend user interfaces to backend servers and databases. They have the skills to build complete web applications end-to-end.

The frontend of our application uses React, which handles the user interface and interactions. However, React alone cannot connect directly to databases or handle server-side logic.

This is where our backend stack comes in. We'll use:

- Node.js (with Typescript) as our server-side runtime environment
- Express.js as our web application framework to handle routing and middleware
- TypeORM as our Object-Relational Mapping (ORM) tool to interact with MySQL



We chose Node.js and Express.js because they use JavaScript / Typescript, allowing us to maintain the same programming language across our entire stack. This provides consistency and reduces the learning curve.

TypeORM will help us interact with our MySQL database using JavaScript / Typescript objects and methods rather than writing raw SQL queries. This makes database operations more intuitive and maintainable.

Activity 1

For activity 1, let's explore the structure of a full stack application

We have two components:

- Our frontend React codebase (the frontend folder)
- Our backend NodeJS codebase (the node-express-typeorm folder)

Backend (node-express-typeorm folder)

Understanding package.json

The `package.json` file contains several important dependencies that form the backbone of our Node.js/Express backend application. Here's a breakdown of the key packages:

1. Core Dependencies:

- `express` (^4.18.3): The main web application framework that provides routing, middleware, and HTTP utilities
- `mysql2` (^3.9.2): A MySQL client for Node.js that enables database connections and queries
- `typeorm` (^0.3.20): An ORM (Object-Relational Mapping) tool that helps manage database operations using TypeScript/JavaScript objects
- `reflect-metadata` (^0.2.1): Required by TypeORM for decorators and metadata reflection

2. Development Dependencies:

- `typescript` (^5.3.3): The TypeScript compiler and language
- `@types/express` (^4.17.21): TypeScript type definitions for Express
- `@types/node` (^20.11.24): TypeScript type definitions for Node.js
- `@types/cors` (^2.8.17): TypeScript type definitions for CORS middleware

3. Development Tools:

- `nodemon` (^3.1.0): Automatically restarts the server when code changes are detected
- `ts-node` (^10.9.2): Enables running TypeScript files directly without pre-compilation

4. Scripts:

- `start`: Runs the compiled JavaScript code (`node dist/index.js`)
- `dev`: Runs the TypeScript code directly with nodemon for development (`nodemon src/index.ts`)
- `build`: Compiles TypeScript to JavaScript (`tsc`)

data-source.ts

The `data-source.ts` file configures the connection between your Node.js application and the MySQL database using TypeORM. It:

1. Sets up the database connection with:

- MySQL database details (host, port, credentials)

- Development settings (auto-sync tables, query logging)
- Entity definitions (currently just the User entity)

2. Provides important features:

- Automatic table creation/updates during development
- SQL query logging for debugging
- TypeORM integration with your entities

Note: The `synchronize: true` setting should be disabled in production to prevent accidental data loss. This setting automatically creates and updates database tables based on your entity definitions. While convenient during development, it can be dangerous in production as it might modify your database schema without explicit control.

Routes

The `routes` folder contains the API endpoint definitions for our application. Each route file defines the HTTP endpoints for a specific resource (like users, products, etc.) and maps them to their corresponding controller methods.

For example, in `user.routes.ts`, we define five main endpoints:

- GET `/users` - Retrieves all users
- GET `/users/:id` - Retrieves a specific user by ID
- POST `/users` - Creates a new user
- PUT `/users/:id` - Updates an existing user
- DELETE `/users/:id` - Deletes a user

These routes use Express Router to handle HTTP requests and delegate the actual business logic to the corresponding controller methods.

Controllers

The `controller` folder is a crucial component of the backend architecture that follows the MVC (Model-View-Controller) pattern. Here's a detailed explanation:

Purpose:

- Controllers act as the middle layer between routes and models / entities
- They handle the business logic of the application
- They process incoming requests and return appropriate responses
- They interact with the database through repositories

Structure:

- Each controller typically corresponds to a specific resource (e.g., UserController for user-related operations)
- Controllers are organized as classes with methods that handle specific HTTP operations
- They use TypeORM repositories to interact with the database

Example (from UserController.ts): The UserController demonstrates typical controller methods:

- `all()`: Retrieves all users
- `one()`: Gets a single user by ID
- `save()`: Creates a new user
- `remove()`: Deletes a user
- `update()`: Updates an existing user

Entities / Models

The `entity` folder contains the data models that define the structure of your database tables. These models are implemented using TypeORM decorators and follow the Entity pattern.

- Each entity is a TypeScript class decorated with `@Entity()`
- Properties are decorated with TypeORM column decorators
- We can use decorators to add special behaviours to our columns
 - `@PrimaryGeneratedColumn()`: Auto-incrementing primary key
 - `@Column()`: Regular database column
 - `@CreateDateColumn()`: Automatically set creation timestamp
 - `@UpdateDateColumn()`: Automatically updated timestamp

Example (from `User.ts`): The User entity demonstrates typical entity structure:

```
@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  firstName: string;

  @Column()
  lastName: string;

  @Column({ unique: true })
  email: string;

  @Column()
  age: number;

  @CreateDateColumn()
  createdAt: Date;

  @UpdateDateColumn()
  updatedAt: Date;
}
```

Frontend (frontend folder)

Our frontend is based on a standard Next.JS app (created using create-next-app)

The **pages** and **components** folders are fairly self explanatory, however, let's have a look at our **services** folder.

This is where we link our API to our frontend.

Services

The **services** folder contains the API integration layer of our frontend application. The **api.ts** file is particularly important as it handles all communication with our backend server. Here's how it works:

1. API Client Setup:

```
export const api = axios.create({
  baseURL: "http://localhost:3001/api",
});
```

- Creates an Axios instance with a base URL
- Centralizes API endpoint configuration
- Makes it easy to change the backend URL in one place

2. Type Definitions:

```
export interface User {
  id: number;
  firstName: string;
  lastName: string;
  email: string;
  age: number;
}
```

- Defines TypeScript interfaces for API data
- Ensures type safety when working with API responses
- Matches the backend entity structure

3. API Methods: The **userApi** object provides methods for all CRUD operations:

- **getAllUsers()**: Fetches all users
- **getUserById(id)**: Gets a specific user
- **createUser(user)**: Creates a new user
- **updateUser(id, user)**: Updates an existing user
- **deleteUser(id)**: Deletes a user

4. Usage in Components:

```
// Example usage in a React component  
const users = await userApi.getAllUsers();
```

- Components can import and use these methods
- Handles all HTTP request/response logic
- Provides consistent error handling
- Returns typed data for type-safe usage